

Software Requirements Specification

for

AflAlert

Version 1.0

Prepared by

Group Name: GROUP 4

NABACHWA PRISCILLA LUTAYA	25/U/0350	<i>priscillanabachwa4@gmail.com</i>
NALUYIMA LAUREEN	25/U/03511/PS	<i>naluyimalaureen@gmail.com</i>
ATWINE SHEILA BAGUMIRE	25/U/03348/EVE	<i>atwiinesheila02@gmail.com</i>
NSUBUGA BRIAN	25/U/03546/PSA	<i>Briannsubuga30@gmail.com</i>
KIZITO JUMA	25/U/26620	<i>Kizitojuma118@gmail.com</i>

Mentor: MR. ASIIMWE PADDY

Course: CSC 1304 Practical Skills Development

Date: 1 August 2026

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Group 4	Initial SRS draft covering introduction, overall description, specific requirements, and non-functional requirements.	1 August 2026

Contents

1 Introduction.....	5
1.1 Document Purpose.....	5
1.2 Product Scope	5
1.3 Intended Audience and Document Overview	5
1.4 Definitions, Acronyms and Abbreviations.....	6
1.5 Document Conventions.....	6
1.6 References and Acknowledgments	6
2 Overall Description	8
2.1 Product Perspective	8
2.2 Product Functionality	8
2.3 Users and Characteristics	9
2.4 Operating Environment.....	9
2.5 Design and Implementation Constraints.....	9
2.6 User Documentation	10
2.7 Assumptions and Dependencies	10
3 Specific Requirements	11
3.1 External Interface Requirements	11
3.1.1 User Interfaces	11
3.1.2 Hardware Interfaces.....	11
3.1.3 Software Interfaces.....	11
3.1.4 Communications Interfaces	11
3.2 Functional Requirements	11
3.2.1 Kernel Image Scanning	11

3.2.2 Test Strip Scanning	12
3.2.3 Guided Recommendations.....	12
3.2.4 Weather and Rain Alerts	12
3.2.5 Voice Assistant	13
3.2.6 History, Reports and Accounts.....	13
3.3 Behaviour Requirements.....	13
3.3.1 Use Case View	13
4 Other Non-functional Requirements.....	15
4.1 Performance Requirements	15
4.2 Safety and Security Requirements	15
4.2.1 Safety.....	15
4.2.2 Security.....	15
4.3 Software Quality Attributes	16
4.3.1 Reliability.....	16
4.3.2 Portability	16
4.3.3 Usability.....	16
4.3.4 Maintainability	16
4.3.5 Availability	16
5 Other Requirements.....	17

1 Introduction

This document is the Software Requirements Specification (SRS) for AflAlert, a mobile application that gives maize farmers and grain handlers a first-line aflatoxin contamination risk check using only a smartphone camera. It was prepared by Group 4 as part of CSC 1304 Practical Skills Development.

The remainder of this section defines the document's purpose and scope, its intended readers, the terminology used throughout, and the conventions and references that apply to the rest of the document.

1.1 Document Purpose

This SRS specifies the functional and non-functional requirements for AflAlert version 1.0. It covers the full application: on-device maize kernel scanning, lateral-flow test-strip reading, guided recommendations, weather/rain alerts, the voice assistant, scan history and PDF reporting, and account management. It is intended to be the shared reference that Group 4 designs, implements, and tests against for the duration of the project, and that the course mentor and facilitator use to assess whether the delivered app meets its stated requirements.

1.2 Product Scope

AflAlert lets a farmer or grain handler photograph a sample of maize kernels, or a reacted lateral-flow test strip after an on-site extraction, and receive an instant, on-device risk read together with guided next steps sourced from UNBS and MAAIF standards. It is designed for the point of harvest or sale, where laboratory testing is rarely available, too slow, or too costly.

Beyond the two scanning tiers, AflAlert helps prevent contamination before it happens: proactive rain and weather alerts warn farmers in time to cover or move drying grain, which is one of the most common triggers of mould growth. Scan history, shareable PDF reports, a hands-free voice assistant, and English/Luganda localization make the app usable and useful for smallholder farmers, grain traders, and the extension officers and buyers they share results with. The product contributes to SDG 2 (Zero Hunger) and SDG 3 (Good Health and Well-Being) by improving food safety at the grassroots level.

1.3 Intended Audience and Document Overview

This document is written for three audiences: the Group 4 development team, who use it as the working reference for what to build and test; the course mentor (Mr. Asiimwe Paddy) and facilitator, who use it to evaluate whether the project meets its stated requirements; and, indirectly, the farmers, grain handlers, and extension officers the product is designed for, whose needs are reflected in Sections 2 and 3.

Section 1 (this section) introduces the document. Section 2 gives an overall description of the product, its users, and the environment it runs in. Section 3 lists specific interface and functional requirements, plus the use-case view. Section 4 covers non-functional requirements (performance, safety/security, quality attributes). Section 5 covers other requirements not captured elsewhere. Readers new to the project should read Sections 1 and 2 first; developers implementing a specific feature should go straight to the relevant subsection of Section 3.

1.4 Definitions, Acronyms and Abbreviations

Aflatoxin: A toxic compound produced by *Aspergillus flavus* and *Aspergillus parasiticus* moulds; classified as a Group 1 human carcinogen by IARC.

App Check: Firebase App Check — verifies that backend requests originate from the genuine AflAlert app.

FR: Functional Requirement.

MAAIF: Ministry of Agriculture, Animal Industry and Fisheries (Uganda).

NFR: Non-Functional Requirement.

OTP: One-Time Password, used during registration and password reset.

ppb: Parts per billion — the unit used for the estimated aflatoxin contamination reading.

SRS: Software Requirements Specification (this document).

Tier 1: The kernel-image scanning detection tier.

Tier 2: The lateral-flow test-strip scanning detection tier.

TFLite: TensorFlow Lite — the on-device machine-learning runtime used for kernel classification.

UI: User Interface.

UNBS: Uganda National Bureau of Standards.

WHO: World Health Organization.

1.5 Document Conventions

This document follows the CSC 1304 SRS template, itself adapted from IEEE requirements-document practice. Section and subsection headings follow the template's numbering. Functional requirements are labelled FR-*area*-*n* and grouped by functional area (Section 3.2); non-functional requirements are referenced by section (e.g. §4.1 for performance). Text in *red italics* marks information that is not yet finalised (e.g. a teammate's e-mail, or meeting minutes still to be logged) and must be filled in by the group before final submission.

1.6 References and Acknowledgments

[1] Google, "Flutter documentation," Flutter.dev. [Online]. Available: <https://flutter.dev/docs>.

[2] Google, "Firebase documentation," Firebase.google.com. [Online]. Available: <https://firebase.google.com/docs>.

[3] Dart team, "Dart language guides," Dart.dev. [Online]. Available: <https://dart.dev/guides>.

[4] Google, "TensorFlow Lite," TensorFlow.org. [Online]. Available: <https://www.tensorflow.org/lite>.

- [5] "image," Dart package, pub.dev. [Online]. Available: <https://pub.dev/packages/image>.
- [6] Open-Meteo, "Weather Forecast API," open-meteo.com. [Online]. Available: <https://open-meteo.com/en/docs>.
- [7] Codex Alimentarius Commission, "General standard for contaminants and toxins in food and feed," CODEX STAN 193-1995, FAO/WHO, Rome, Italy, 1995 (rev. 2019).
- [8] Uganda National Bureau of Standards (UNBS), "Maize grains — Specification," Kampala, Uganda. [Online]. Available: <https://www.unbs.go.ug>.
- [9] Ministry of Agriculture, Animal Industry and Fisheries (MAAIF), "Post-harvest handling and grain storage guidelines," Kampala, Uganda. [Online]. Available: <https://www.agriculture.go.ug>.
- [10] World Health Organization, "Aflatoxins," WHO Food Safety Digest, 2023.
- [11] International Agency for Research on Cancer, "Aflatoxins," IARC Monographs, 2012.
- [12] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," New York: UN, 2015.
- [13] "flutter_tts," Dart package, pub.dev. [Online]. Available: https://pub.dev/packages/flutter_tts.
- [14] "workmanager," Dart package, pub.dev. [Online]. Available: <https://pub.dev/packages/workmanager>.

2 Overall Description

2.1 Product Perspective

AfIAlert is a new, self-contained mobile product, not a follow-on to an existing system. It is a cross-platform Flutter application (Android and iOS, with a limited Firebase-hosted web build) backed by Firebase for authentication, data storage, and server-side logic. Kernel classification runs entirely on-device via a bundled TensorFlow Lite model, so Tier 1 scanning works without a network connection; test-strip reading is also on-device, using deterministic image processing rather than a trained model. Account sync, scan-history storage, PDF report generation/sharing, and weather-based alerts depend on connectivity to Firebase and the Open-Meteo weather API.

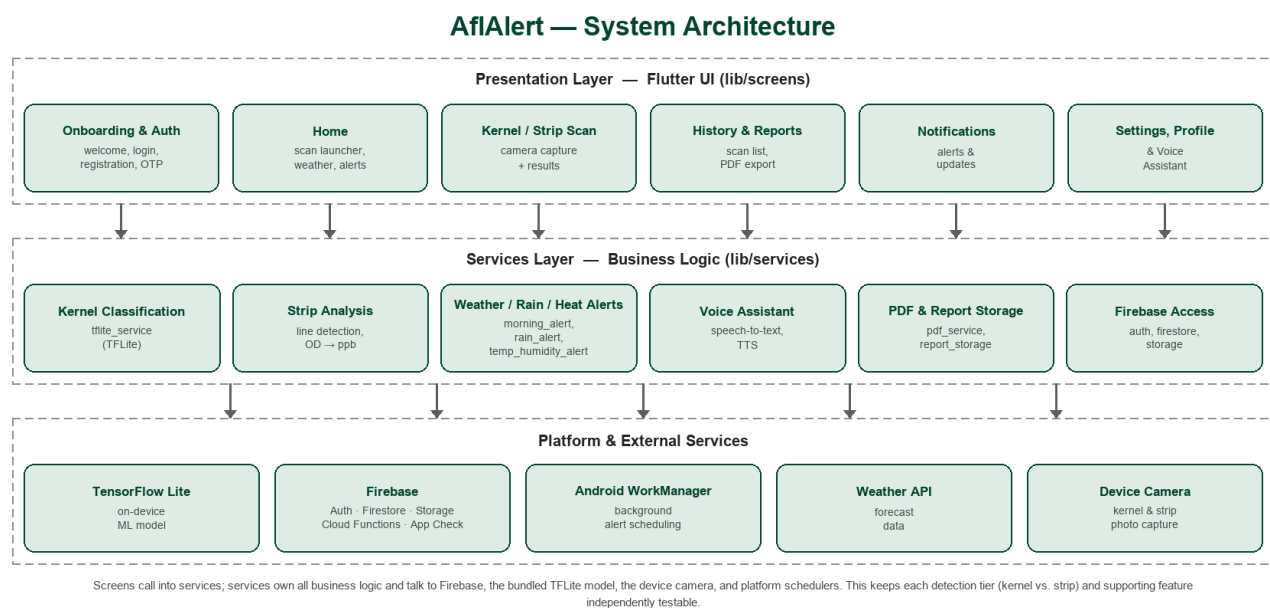


Figure 1: AfIAlert system context — Flutter UI screens call into a services layer (classification, strip analysis, alerts, voice, PDF/report storage, Firebase access), which talks to the platform logic and external services below it.

2.2 Product Functionality

- Kernel scan (Tier 1) — photograph maize kernels; an on-device TFLite model flags visible mould/discoloration risk.
- Test strip scan (Tier 2) — photograph a reacted lateral-flow strip; the app reads the Control/Test lines and estimates a ppb contamination reading.
- Guided recommendations — every result comes with UNBS/MAAIF-sourced next-step guidance.
- Weather and rain alerts — a daily morning summary plus urgent incoming-rain alerts so drying grain can be protected.
- Voice assistant — hands-free scanning and Q&A, reachable from the home, history, notifications, and result screens.

- Scan history and PDF reports — review past scans and export/share a PDF report per result.
- Accounts — email/Google sign-in, OTP verification, password reset, biometric app unlock.
- Localization — full English and Luganda support throughout the app.

2.3 Users and Characteristics

a) Farmers and grain handlers (primary): Vary widely in literacy, smartphone experience, and connectivity. Use kernel scanning most, benefit most from the voice assistant and Luganda localization, and are the group the app's core UX decisions are optimised for.

b) Grain traders and buyers (secondary): Use scan results and PDF reports as a quick, shareable quality signal before a purchase or sale; more likely to also use Tier 2 strip scanning where a more quantitative reading matters.

c) Extension officers and inspectors (tertiary): Consume shared PDF reports and guidance content rather than performing scans themselves; interact with the app mainly at the point results are shared with them.

d) Group 4 (developers/administrators): Maintain the Firebase backend, Cloud Functions, and the bundled TFLite model; the only users with access to backend configuration, not represented in the end-user use cases in §3.3.

Farmers and grain handlers are the most important user group to satisfy: the kernel-scan tier is deliberately offline-capable and camera-only so it works for them regardless of connectivity or access to a test strip.

2.4 Operating Environment

Client: Android (minSdk 24, i.e. Android 7.0+) and iOS, built with Flutter (Dart SDK ^3.12.2). A Firebase Hosting web build (<https://aflalert.web.app>) supports onboarding, authentication, history, and alerts, but not camera-based scanning, since the bundled TFLite classifier relies on dart:ffi native bindings with no browser-compatible implementation.

Backend: Firebase Authentication, Cloud Firestore, Firebase Storage, Cloud Functions, and Firebase App Check, all hosted on Google Cloud. Weather data is fetched from the Open-Meteo forecast API. Background alert scheduling on Android uses WorkManager so alerts can fire while the app is closed.

2.5 Design and Implementation Constraints

a) On-device ML via dart:ffi: The kernel classifier is a bundled TFLite model accessed through `tflite_flutter/dart:ffi`, which has no browser-compatible implementation — camera-based scanning is therefore Android/iOS-only, not available on the web build.

b) Firebase platform dependency: Authentication, data storage, file storage, and backend logic are built directly on Firebase's Auth/Firestore/Storage/Functions/App Check services, which constrains hosting choice and offline-sync behaviour to what those SDKs provide.

c) Android-only background scheduling: WorkManager provides no equivalent timing guarantee on iOS, so proactive rain/weather alerts currently only run reliably on Android.

d) Unvalidated ppb calibration: The strip reader's ppb figure is computed from a monotonic placeholder curve (not a lab-fitted dose-response curve), constraining Tier 2 results to a relative/qualitative signal until certified reference strips are used to calibrate it.

e) Cross-platform language/framework: The team committed to Flutter/Dart for a single codebase across Android, iOS, and web, and to maintaining full English/Luganda parity (app_localizations ARB files) for every user-facing string.

f) Course timeline and team size: Delivered by a 5-person team within the CSC 1304 Practical Skills Development timeline, which bounds how much of the roadmap in §3.2 and the technical report's "Next Steps" can be completed before submission.

2.6 User Documentation

AflAlert relies on in-app guidance rather than a separate manual: an onboarding carousel introduces the app's purpose at first launch, and guided capture overlays walk the user through kernel and strip photo capture at the point of use. A short online FAQ/user guide is planned alongside the GitHub repository for users who want more detail than the in-app guidance provides. All help content is localized in English and Luganda, consistent with the rest of the app.

2.7 Assumptions and Dependencies

- Users have an Android or iOS smartphone with a working rear camera.
- Kernel scanning (Tier 1) works offline, but account sync, history storage, weather alerts, and PDF sharing assume at least intermittent internet connectivity.
- Firebase's free/Blaze-tier services remain available and sufficient for the project's usage volume.
- Third-party packages the app depends on (tflite_flutter, image, workmanager, flutter_tts, geolocator) remain compatible with the Dart SDK ^3.12.2 constraint.
- The Open-Meteo forecast API remains accessible and reasonably accurate for the Ugandan districts the app targets.
- Users performing a Tier 2 scan have access to a commercially available lateral-flow aflatoxin test strip and can follow the extraction step correctly.
- The Tier 2 ppb reading is assumed to be a relative/qualitative signal, not a certified quantitative assay result, until it is calibrated against certified reference standards.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

AflAlert's screens fall into four groups: onboarding/authentication (onboarding carousel, welcome, login, registration, OTP verification, forgot-password), the home screen (scan launch buttons, recent scans, weather/alert banner), the two scan flows (guided capture → result, for both kernel and strip), and supporting screens (history, notifications, voice assistant overlay, settings, profile). Every screen exposes a consistent back/cancel action; capture screens show an in-frame guide overlay; result screens use a consistent safe/unsafe colour and iconography; invalid or low-confidence scans are shown as an inline error/retry banner rather than a silent failure or a misleading result.

3.1.2 Hardware Interfaces

- Rear camera — used for kernel and test-strip photo capture (camera / image_picker).
- Location sensor — device GPS/network location (geolocator, geocoding) used to resolve the user's district for weather and rain alerts.
- Biometric hardware — fingerprint/face unlock (local_auth) used for optional app-unlock on a signed-in session.
- Touchscreen — the sole input mechanism outside of the voice assistant's microphone input.

3.1.3 Software Interfaces

- Android and iOS platform APIs (camera, notifications, biometrics, background execution).
- Firebase SDKs — Authentication, Cloud Firestore, Storage, Cloud Functions, App Check.
- TensorFlow Lite runtime (tflite_flutter) for on-device kernel classification.
- WorkManager (Android) for background rain/weather alert scheduling.
- flutter_local_notifications for displaying alerts and scan-complete notifications.

3.1.4 Communications Interfaces

All communication with the Firebase backend and Cloud Functions is over HTTPS, authenticated with a Firebase Auth session token and validated with a Firebase App Check token to reject requests not originating from the genuine app. Weather data is fetched over HTTPS from the Open-Meteo REST API using the device's resolved coordinates. Push/local notifications carry the daily weather summary and urgent rain alerts. PDF reports are handed off to the platform share sheet (e-mail, WhatsApp, etc.) rather than transmitted by the app itself. No custom encryption is implemented beyond standard TLS in transit and the security controls already provided by Firebase.

3.2 Functional Requirements

3.2.1 Kernel Image Scanning

ID	Requirement
FR-KER-1	The user shall be able to capture a photo of a maize kernel sample via the in-app camera, or select an existing photo from the gallery.
FR-KER-2	The system shall classify the captured/selected image on-device via the bundled TFLite model (e.g. healthy vs. mouldy/spoiled) without requiring a network connection.
FR-KER-3	The system shall reject low-confidence or clearly non-maize images (colour-profile mismatch, model rejection, low confidence) instead of returning a result.

3.2.2 Test Strip Scanning

ID	Requirement
FR-STR-1	The system shall present a guided capture frame aligning the Control line near the top and Test line near the bottom, and check light/focus quality before capture.
FR-STR-2	The system shall reject photos that do not plausibly resemble a strip cassette before attempting line detection.
FR-STR-3	The system shall locate the Control (C) and Test (T) line bands from a row-by-row luminance profile, using a local-background baseline for robustness to uneven lighting.
FR-STR-4	The system shall compute the optical density of each line relative to its local background, derive a Test/Control ratio, and translate it into an estimated ppb reading, classified safe when at or below 20 ppb.
FR-STR-5	The system shall flag a scan as invalid (rather than presenting a misleading low-ppb result) if no strip/lines are detected, or if the Control line is not dark enough to indicate a properly developed run.

3.2.3 Guided Recommendations

ID	Requirement
FR-REC-1	The system shall attach next-step guidance, sourced from UNBS and MAAIF standards, to every scan result (kernel or strip), based on whether the

	result is within or above the applicable safety threshold.
--	--

3.2.4 Weather and Rain Alerts

ID	Requirement
FR-ALT-1	The system shall provide a once-daily weather summary based on the user's resolved location.
FR-ALT-2	The system shall independently monitor short-range forecasts and push an urgent alert when rain is imminent, including while the app is closed (Android background scheduling).

3.2.5 Voice Assistant

ID	Requirement
FR-VOI-1	The user shall be able to trigger scans and ask questions about results/recommendations using speech input, with spoken responses.
FR-VOI-2	The assistant shall be reachable from the home, history, notifications, and both result screens.
FR-VOI-3	When a scan is triggered from the assistant, the system shall speak a summary of the result (safe/unsafe) via text-to-speech as soon as it is ready, using the same label logic as the on-screen result.

3.2.6 History, Reports and Accounts

ID	Requirement
FR-HIS-1	The system shall store and display a history of past kernel and strip scans per authenticated user.
FR-HIS-2	The user shall be able to generate and share/download a PDF report for a given scan result.
FR-ACC-1	The system shall support email/password and Google sign-in, OTP verification, password reset, and biometric app unlock.
FR-ACC-2	The system shall support English and Luganda throughout the app, switchable from Settings.

3.3 Behaviour Requirements

3.3.1 Use Case View

The primary actor is the Farmer / Grain Handler, who drives every use case except the alert-triggering itself. The Background Scheduler (Android WorkManager) is a secondary actor that independently triggers the “Receive Weather & Rain Alerts” use case even while the app is closed. Use cases:

- Register / Sign In — create an account or sign in (email/password or Google), including OTP verification.
- Scan Maize Kernels (Tier 1) — capture/select a kernel photo and get an on-device risk classification.
- Scan Test Strip (Tier 2) — capture a reacted strip photo and get a ppb-based risk reading.
- View Guided Recommendations — view UNBS/MAAIF-sourced next steps attached to a scan result.
- Receive Weather & Rain Alerts — receive the daily summary and urgent rain notifications.
- Use Voice Assistant — trigger scans and ask result/recommendation questions hands-free.
- View Scan History — browse past kernel and strip scan results.
- Export PDF Report — generate and share a PDF of a scan result.
- Manage Settings & Profile — change language, notification preferences, and account details.

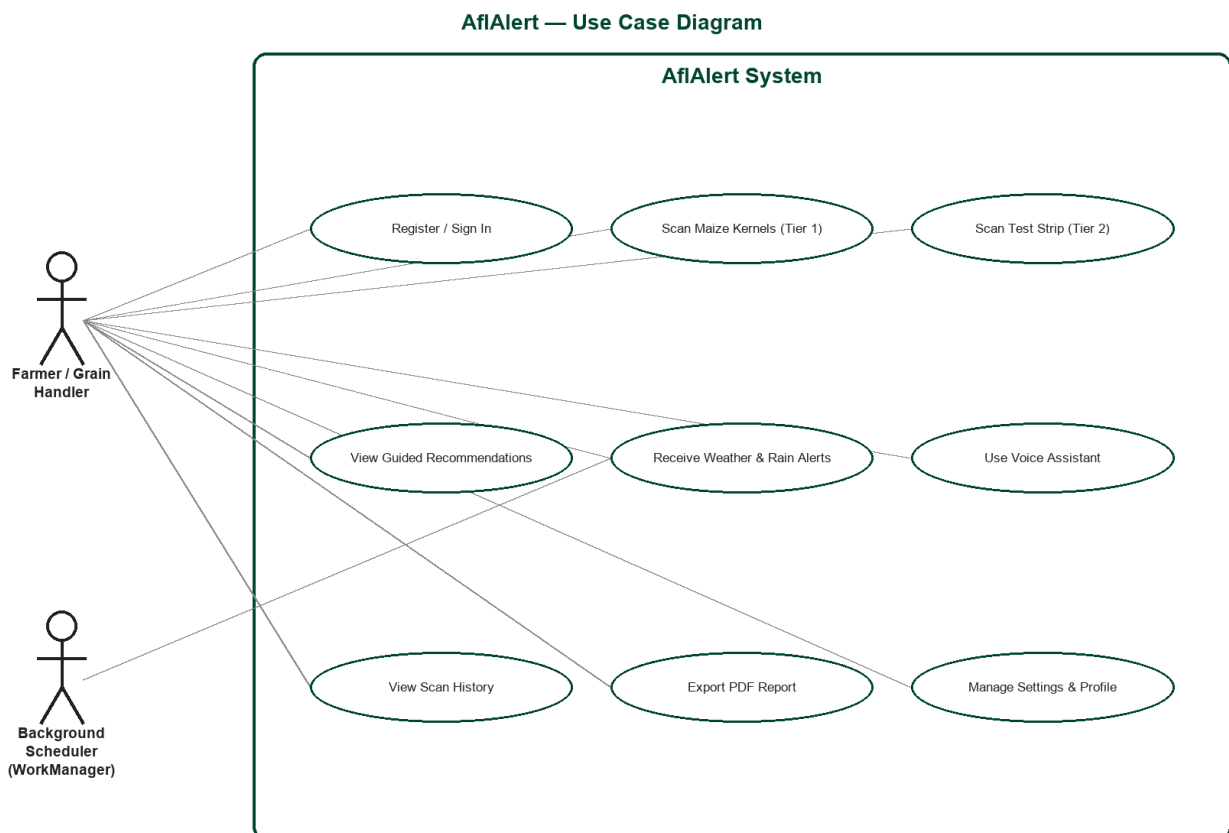


Figure 2: AfIAlert use case diagram — Farmer/Grain Handler as primary actor against all end-user use cases; Background Scheduler as secondary actor triggering rain/weather alerts independently of the app being open.

4 Other Non-functional Requirements

4.1 Performance Requirements

1. Kernel scan classification shall return a result within 3 seconds of capture on a mid-range device, since inference runs on-device with no network round-trip.
2. Test-strip line detection and ppb calculation shall complete within 5 seconds of capture.
3. The app shall reach the home screen within 4 seconds of a cold start on a mid-range Android device.
4. A scan-history list of up to 200 entries shall load within 2 seconds from Firestore under normal connectivity.
5. PDF report generation and hand-off to the platform share sheet shall complete within 3 seconds of the user's request.
6. The daily weather summary notification shall be delivered within a fixed morning window (e.g. 6:00–8:00 AM local time) on at least 95% of scheduled days.

4.2 Safety and Security Requirements

4.2.1 Safety

7. Every result screen (kernel or strip) shall display a disclaimer that the reading is a first-line risk indicator, not a certified laboratory diagnosis, given the seriousness of chronic aflatoxin exposure.
8. Results above the applicable safety threshold shall carry explicit “do not mix with clean grain” / “set aside for inspection” guidance, to prevent contaminated grain re-entering the food supply.
9. Invalid or low-confidence scans (poor lighting, undetected lines, non-maize image) shall be rejected rather than silently returned as a “safe” result.
10. High-risk (above-threshold) results shall recommend confirmatory laboratory testing rather than being presented as a final determination.

4.2.2 Security

- Scan history and reports are stored per-account and require the user to be authenticated (email/password or Google sign-in).
- Registration and password reset require OTP verification.
- Biometric app-unlock (local_auth) is available to protect an active session on a shared device.
- Firebase App Check verifies that backend requests originate from the genuine AflAlert app.
- All network traffic is transmitted over HTTPS/TLS.
- Firestore security rules restrict each user to reading and writing only their own scan history and profile data.
- No scan photos or location data are shared with third parties beyond what is required for weather lookups.

4.3 Software Quality Attributes

4.3.1 Reliability

Kernel scanning (Tier 1) works fully offline, so the app's core function is not affected by connectivity loss. Both scan tiers detect and reject invalid input (low confidence, undetected lines, poor-quality capture) rather than failing silently or returning a misleading result.

4.3.2 Portability

A single Flutter/Dart codebase targets Android, iOS, and a limited Firebase-hosted web build, sharing UI, services, and localization resources across all three.

4.3.3 Usability

Guided capture overlays, a hands-free voice assistant, and full English/Luganda localization are prioritised specifically because the primary users — smallholder farmers and grain handlers — vary widely in literacy and smartphone experience. Ease of use is prioritised over feature breadth for this group.

4.3.4 Maintainability

The codebase separates screens (UI), services (business logic — classification, strip analysis, Firebase access, alerts, PDF generation, voice), and localization resources, so each detection tier and supporting feature can be modified and tested independently.

4.3.5 Availability

Backend-dependent features (sync, history, alerts, PDF sharing) inherit Firebase's published service availability; the core kernel-scan feature remains available even during a backend outage, since it does not depend on the network.

5 Other Requirements

a) Localization: Full English and Luganda parity is required across all user-facing strings (app_localizations ARB files); the localization architecture should allow additional languages to be added without changing screen code.

b) Legal / data handling: A Terms of Service and data-handling disclosure covering photo and location storage shall be presented at sign-up.

c) Data retention: Scan photos and history are retained per-account until the user deletes their account, at which point associated Firestore and Storage data shall be removed.

Appendix B - Group Log

Date	Attendees	Summary / Decisions
22/06/26	All group members	Meeting 1: Project Kickoff & Environment Setup - Installed Flutter, Android Studio, and set up the shared GitHub repository. Requirements review and literature study on aflatoxin detection methods. Assigned roles to each team member. Discussed project scope (maize-only detection) and deliverables.
25/06/26	All group members	Meeting 2: Dataset Collection Planning - Agreed on maize image sourcing strategy: on-site collection from Mukono plantations, roadside maize sellers, and maize traders, supplemented with public datasets (Makerere AI Lab, Harvard Dataverse) and Roboflow. Collection began immediately after the meeting.
29/06/26	All group members	Meeting 3: First Training Run - Reviewed the first batch of collected maize images. Began MobileNetV2 transfer-learning experiments on Google Colab. Agreed that dataset collection and retraining would continue as an ongoing cycle throughout the project: new images gathered in the field or online were added to the dataset and the model retrained whenever a meaningful batch was ready.
03/07/26	All group members	Meeting 4: App Features & Individual Role Assignment - Discussed the full set of app features and screens. Assigned individual development responsibilities: Nsubuga Brian - history screen and Luganda localization; Naluyima Laureen - app logo and login screen;

		Nabachwa Priscilla - settings, splash and onboarding screens, strip scan feature, and weather feature; Atwine Sheila - notifications, results screen, registration, and profile; Kizito Juma - home screen and overall UI design lead.
08/07/26	All group members	Meeting 5: Continued Data Collection & Retraining - Additional maize images collected and added to the dataset; model retrained on Google Colab with the expanded dataset. Screen development began in parallel per the roles agreed in Meeting 4.
13/07/26	All group members	Meeting 6: Environment & Dataset Setup - Flutter/Firebase project setup finalized. MobileNetV2 transfer-learning research continued on Google Colab. App wireframes reviewed.
15/07/26	All group members	Meeting 7: ML Pipeline & App Progress - TFLite model conversion and quantization started. Core Flutter screens (Home, Results, Settings) development continued. Firebase Auth and Firestore integration begun.
16/07/26	All group members	Meeting 8: UI & Integration - Results screen UI finalized (dark theme, cream cards, forest green accents). Google Sign-In configuration completed. Test cases prepared.
20/07/26	All group members	Meeting 9: Feature Development - Offline kernel detection tested on-device. Luganda language support added. Voice assistant integration (speech-to-text, TTS, Firebase Cloud Function) started. Scan history feature implemented.
22/07/26	Priscilla, Sheila, Juma (field visit); reviewed by all group members	Field Testing - Visited maize farmers at Mukono plantations to

		test the kernel scan feature under real field conditions and gather user feedback on accuracy, usability, and the guided capture flow. Findings reported back to the group and used to refine the capture UI.
23/07/26	All group members	Meeting 10: Lateral Flow Strip Scan - Lateral flow strip scan feature designed and integrated for chemical test strip analysis, providing precise ppb-level aflatoxin readings alongside image-based detection. Feature tested against sample strips.
24/07/26	Priscilla, Sheila (site visit); reviewed by all group members	Stakeholder Engagement - Visited the Uganda National Bureau of Standards (UNBS) to research official standards and testing procedures for aflatoxin test strips. Findings directly informed the strip scan feature's line-detection logic and ppb safety threshold.
25/07/26	All group members	Meeting 11: Website & Documentation - Project website developed, customized, and hosted on GitHub Pages. SRS documentation progressed. App features consolidated and reviewed.
28/07/26	All group members	Meeting 12: Final Review - End-to-end app testing completed. Bug fixes and Gradle build issue resolution. Final report writing and documentation proofreading. All deliverables (poster, report, website, repository) prepared for submission.